

Gestionale Affitti

Relazione finale di Ingegneria del Software

Progetto individuale

Versione 1.0

Indice

1	Obiettivo e scope	2
2	Requisiti e casi d'uso	2
3	Modellazione del dominio	4
4	Architettura	6
5	SOLID e pattern	8
5.1	SRP	8
5.2	DIP	8
5.3	OCP	8
5.4	Pattern non introdotti	8
6	UC-01: registrazione del contratto	8
7	UC-02: registrazione del pagamento	11
8	Persistenza	13
9	Testing	13
10	Quality gate e Continuous Integration	13
11	Tracciabilità	13
12	Trade-off e limiti della versione 1.0	14
13	Conclusioni	14

1 Obiettivo e scope

Gestionale Affitti è un'applicazione web dimostrativa e mono-utente per la gestione essenziale di locazioni immobiliari ad uso abitativo. La versione 1.0 è volutamente limitata a due casi d'uso core, scelti per mantenere il progetto piccolo ma completo e tracciabile:

- **UC-01 – Registrare un contratto di locazione;**
- **UC-02 – Registrare il pagamento di un canone.**

L'unico attore diretto è il proprietario. L'inquilino è un concetto del dominio, ma non interagisce direttamente con il sistema. Restano fuori scope autenticazione, uso pubblico o multiutente, rinnovi operativi, pagamenti parziali, trasferimenti bancari reali, firma e registrazione fiscale del contratto.

La specifica completa e verificabile è mantenuta in `docs/requisiti.tex`. Durante la review finale è stato eliminato il precedente requisito prestazionale quantitativo di 1,5 secondi perché non era accompagnato da condizioni nominali e ambiente di misura riproducibili: è stato preferito rimuovere un requisito non realmente verificabile anziché introdurre un benchmark artificiale.

2 Requisiti e casi d'uso

UC-01 guida il proprietario attraverso cinque step: Immobile, Proprietario, Inquilino, Dati contrattuali e Riepilogo. I primi quattro step acquisiscono dati e aggiornano una bozza persistente; il riepilogo permette di tornare agli step completati, salvare la bozza, annullarla oppure confermare la registrazione definitiva.

UC-02 parte dalla selezione univoca dell'Immobile e dell'Inquilino, individua automaticamente la mensilità non pagata cronologicamente più vecchia fino al mese corrente e propone una preview. La conferma ricarica i dati persistiti, rivalida la competenza, ricalcola l'importo e registra il Pagamento usando la data corrente del server. La preview non è quindi una fonte autorevole.

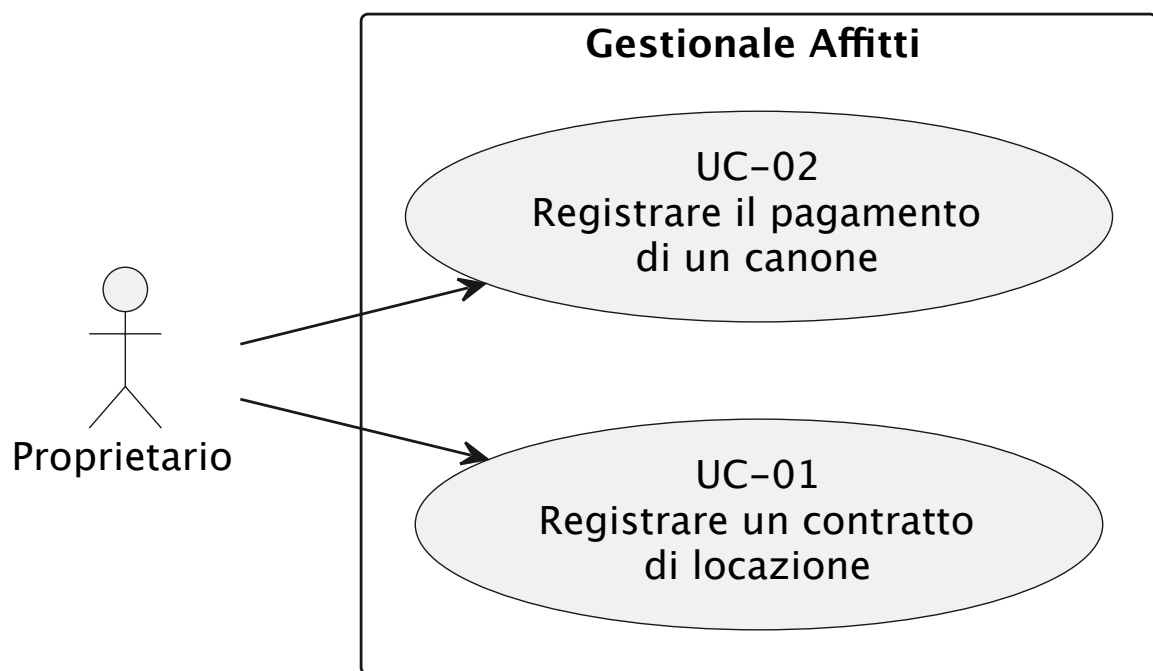


Figura 1: Use Case Diagram della versione 1.0

3 Modellazione del dominio

Il Domain Model mantiene separati i concetti stabili del dominio dai modelli temporanei del workflow applicativo. Le entità e value object principali sono **Persona**, **Immobile**, **Indirizzo**, **DatiCatastali**, **DocumentoRiconoscimento**, **Contratto**, **TipologiaContrattuale**, **Articolo** e **Pagamento**.

BozzaContratto non appartiene al Domain Model: rappresenta lo stato recuperabile della procedura di UC-01. Analogamente, **PagamentoDaRegistrare** è un application model usato per la preview di UC-02. La mensilità non è modellata come entità persistente, perché è derivabile dal periodo del Contratto, dai pagamenti registrati e dalla data corrente.

Le principali invarianti sono:

- proprietario e inquilino devono essere Persone distinte;
- il giorno di pagamento è compreso tra 1 e 28;
- i periodi di Contratti dello stesso Immobile non possono sovrapporsi;
- per lo stesso Contratto può esistere al massimo un Pagamento per anno e mese di competenza;
- l'importo della prima e dell'ultima mensilità parziale è calcolato in pro-rata, con arrotondamento finale a due cifre decimali;
- la data corrente è una dipendenza autorevole del server e non un dato fornito dal client.

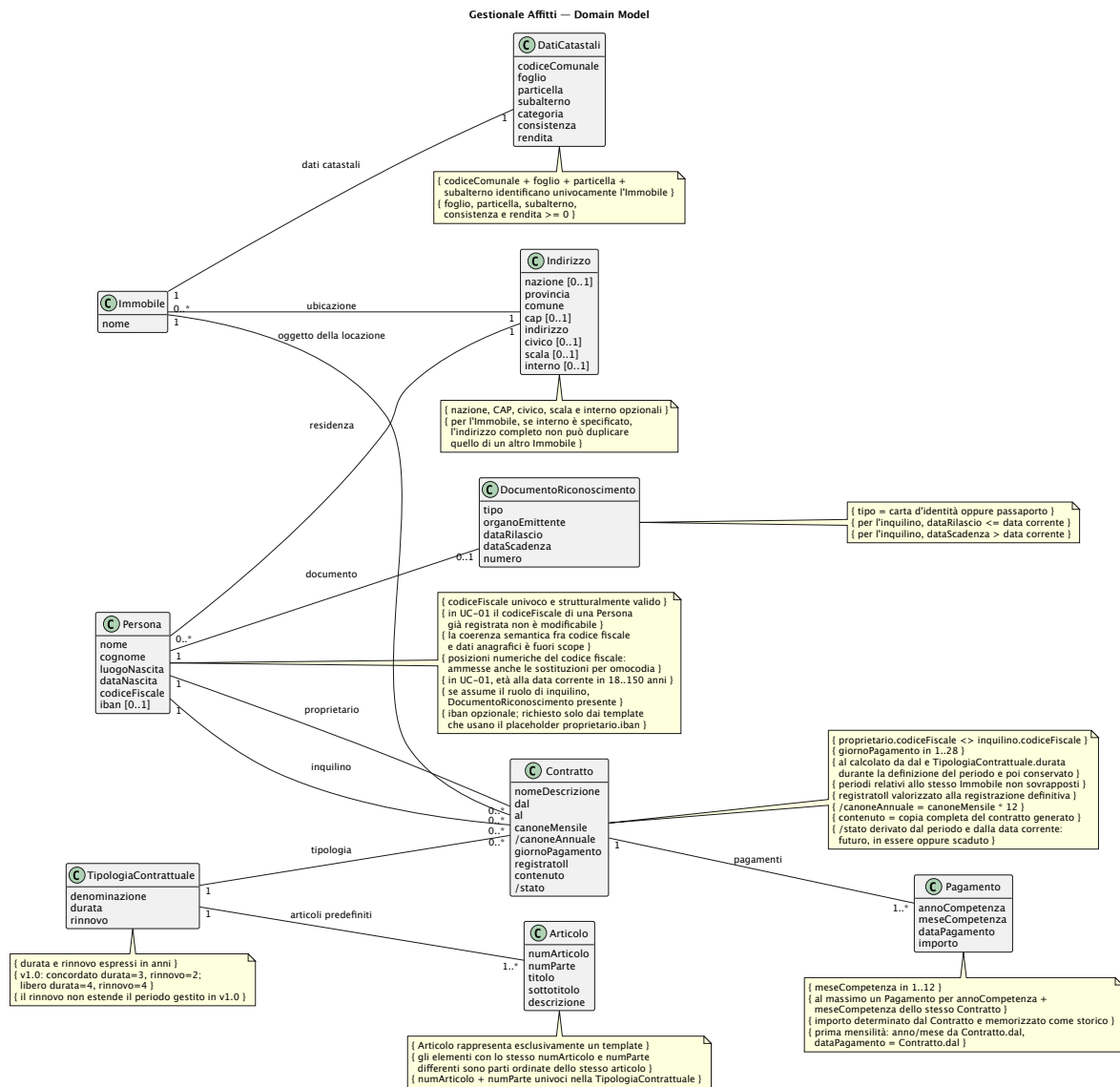


Figura 2: Domain Model

4 Architettura

Il sistema adotta una struttura client-server e, all'interno del backend monolitico, una separazione layered. Le responsabilità logiche sono:

web/interface ricezione delle richieste HTTP, validazione formale, serializzazione e traduzione degli errori;

application orchestrazione dei casi d'uso tramite `RegistraContrattoService` e `RegistraPagamentoService`;

domain regole stabili e invarianti del dominio;

infrastructure PostgreSQL, mapping, transazioni, generazione HTML, data corrente e dettagli tecnici.

L'application layer dipende da porte definite sui bisogni dei casi d'uso. Le implementazioni concrete PostgreSQL dipendono da tali contratti. Il composition root in `src/web/compositionRoot.ts` è l'unico punto che conosce insieme i Service e le implementazioni infrastrutturali.

Per UC-01, `RegistrazioneContrattoPort` delimita l'operazione atomica di registrazione definitiva: dati definitivi collegati, Contratto e primo Pagamento vengono scritti nella stessa transazione PostgreSQL. Il cleanup della bozza è successivo al commit, così un suo eventuale fallimento non invalida un Contratto già registrato.

Per UC-02, `PagamentoRepository` è volutamente una porta di sola scrittura. I pagamenti storici vengono letti insieme al Contratto tramite `ContrattoRepository`; questa asimmetria evita due percorsi concorrenti per ricostruire lo stesso stato.

5 SOLID e pattern

5.1 SRP

Controller, Service, dominio e infrastruttura hanno motivi di cambiamento distinti. I Service non eseguono SQL né rendering concreto; i repository non contengono workflow; il dominio non dipende da Express o PostgreSQL.

5.2 DIP

Le dipendenze sostituibili sono invertite mediante porte: repository, generatore del documento e provider della data corrente. Questo permette di testare la business logic senza database o orologio reale.

5.3 OCP

Non vengono introdotte gerarchie preventive per le tipologie contrattuali, perché nella versione 1.0 differiscono per dati e template, non per famiglie di algoritmi. Il generatore del documento è invece isolato dietro una porta perché il formato di output rappresenta un punto di variazione concreto.

5.4 Pattern non introdotti

Strategy, Factory Method, Adapter e Observer sono stati valutati ma non adottati esplicitamente: nello scope corrente non risolvono un problema reale meglio della soluzione più semplice. Questa scelta evita overengineering e pattern dimostrativi non giustificati.

6 UC-01: registrazione del contratto

Il flusso di UC-01 utilizza bozze persistenti per consentire interruzione e ripresa. Nuovi Immobili, nuove Persone e modifiche alle Persone esistenti rimangono temporanei fino alla conferma. Il sistema valida codice fiscale, età, documento dell'Inquilino, eventuale IBAN richiesto dal template, durata del Contratto e sovrapposizione del periodo.

L'anteprima usa lo stesso generatore documentale della conferma, ma non produce scritture definitive. Alla conferma il Contratto viene ricostruito dalla bozza, i dati temporali vengono rivalidati, il documento viene rigenerato e conservato come contenuto storico. Viene inoltre creato il Pagamento della prima competenza.

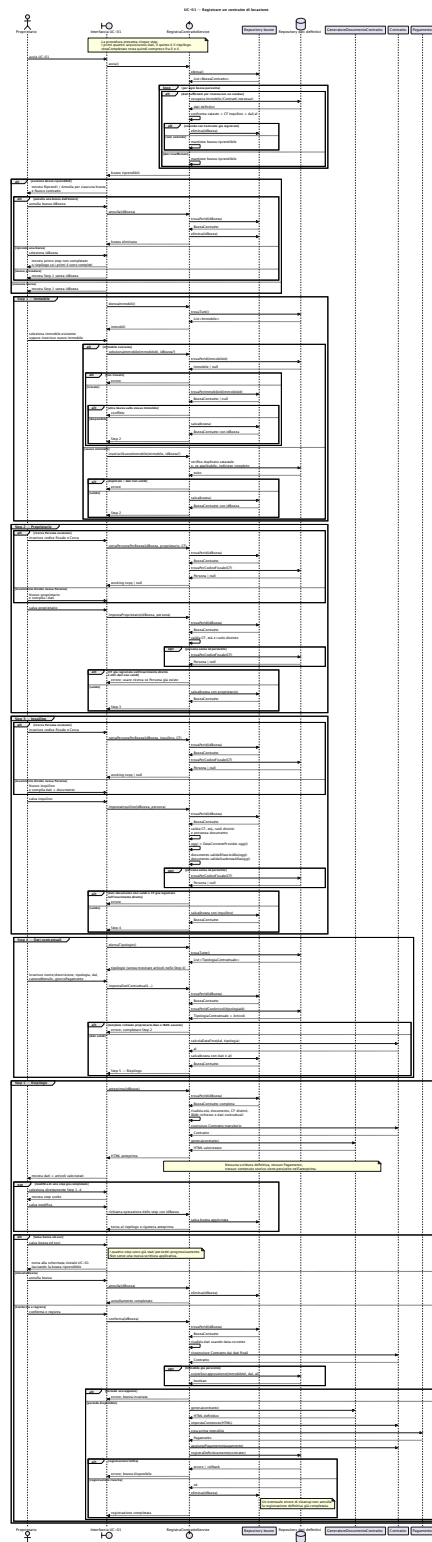


Figura 4: Sequence Diagram di UC-01

7 UC-02: registrazione del pagamento

Il Service carica i Contratti della coppia Immobile-Inquilino, considera i Pagamenti già registrati e individua la prima competenza non pagata fino al mese corrente. Le mensilità future sono escluse; quella corrente è pagabile dal primo giorno anche se non è ancora dovuta.

Il Contratto calcola importo e scadenza. La preview espone anche lo stato dovuta/tardivo, ma alla conferma non viene considerata autorevole: il Service ricarica il Contratto, ricalcola la competenza più vecchia e rifiuta la conferma se la preview è diventata obsoleta. Il database mantiene inoltre un vincolo UNIQUE (`contratto_id`, `anno_competenza`, `mese_competenza`) come protezione secondaria.

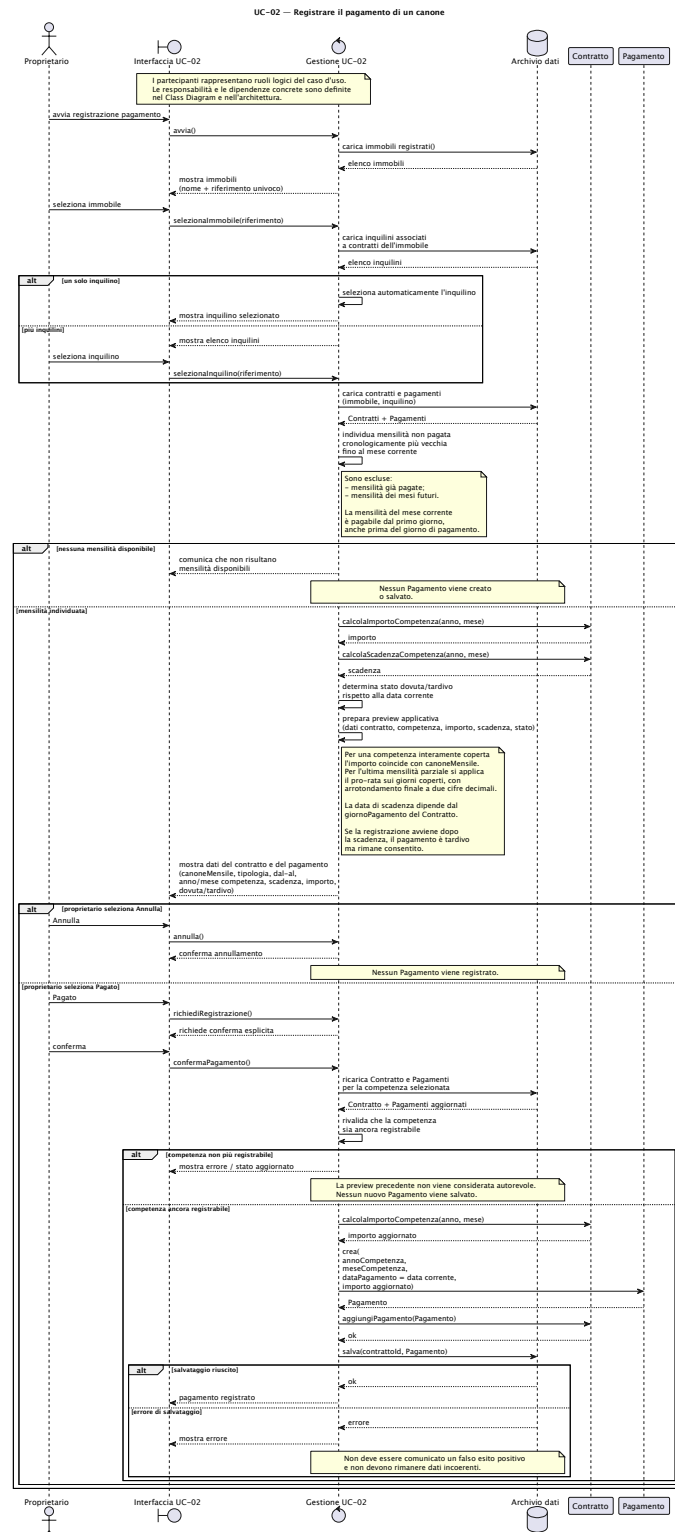


Figura 6: Sequence Diagram di UC-02

8 Persistenza

La persistenza usa PostgreSQL tramite il driver `pg` e SQL esplicito. Le migration sono versionate in `db/migrations/`; i template contrattuali iniziali sono JSON versionati in `db/seed/template/` e caricati con un seed idempotente e transazionale.

I dati definitivi sono relazionali. La bozza di UC-01 è l'unica struttura persistita come JSONB, perché rappresenta uno stato applicativo temporaneo ed eterogeneo. Il contenuto storico del Contratto è memorizzato come HTML in una colonna testuale e non viene rigenerato in seguito dai template correnti.

9 Testing

La strategia di test privilegia gli unit test sulle regole del dominio e sull'orchestrazione applicativa. Fake e stub sono usati per isolare repository e provider della data quando servono realmente; i test PostgreSQL sono riservati a query, mapping, vincoli e transazioni.

Le suite coprono:

- regole di **Contratto**, **Persona**, **DocumentoRiconoscimento**, **DatiCatastali** e **Pagamento**;
- lifecycle delle bozze e conferma di UC-01;
- generazione documentale e placeholder;
- repository e schema PostgreSQL;
- confine HTTP dei due casi d'uso;
- scenario end-to-end di UC-02 attraverso HTTP, application layer e PostgreSQL.

Lo scenario end-to-end di UC-02 verifica sia il flusso positivo sia un errore significativo: una preview diventa obsoleta prima della conferma e viene rifiutata senza creare duplicati o registrare una competenza differente.

10 Quality gate e Continuous Integration

Il comando canonico di verifica è:

```
npm run verify
```

Esso esegue build TypeScript, type-check dei test, ESLint e Jest. La pipeline GitHub Actions riproduce lo stesso quality gate in container, avvia PostgreSQL e usa `npm ci` per rispettare il lockfile. In questo modo la CI non introduce una definizione parallela di “progetto valido”, ma automatizza il controllo già disponibile localmente.

11 Tracciabilità

La tracciabilità completa è mantenuta in `docs/tracciabilita.md`. Per entrambi i casi d'uso è possibile seguire il percorso:

Requisito → **Acceptance Criteria** → **Caso d'uso** → **UML** → **Componente** → **Codice** → **Test**

Questa catena è usata anche come criterio di review: un componente importante deve essere giustificato da una responsabilità o da un requisito, e un requisito core deve avere evidenza implementativa e test significativa.

12 Trade-off e limiti della versione 1.0

Le principali scelte consapevoli sono:

- monolite layered invece di servizi distribuiti, per evitare complessità non richiesta;
- SQL esplicito invece di ORM, perché lo scope rende gestibili query e mapping;
- nessun pattern GoF forzato;
- una porta `PagamentoRepository` orientata alla scrittura per evitare duplicazioni di lettura;
- nessun vincolo prestazionale quantitativo senza ambiente di misura riproducibile;
- contesto mono-utente, senza autenticazione e autorizzazione;
- gestione del solo periodo iniziale del Contratto, senza rinnovi operativi.

Questi limiti mantengono il progetto coerente con lo scope d'esame e rendono esplicito quali aspetti richiederebbero una revisione architetturale in un'evoluzione futura.

13 Conclusioni

La versione 1.0 implementa due casi d'uso completi e coerenti lungo l'intera catena di sviluppo: requisiti, modellazione, architettura, codice, test, CI e documentazione. La scelta progettuale dominante è mantenere responsabilità esplicite e dipendenze sostituibili senza introdurre complessità preventiva. Il risultato è un progetto contenuto, testabile e difendibile durante la discussione orale.